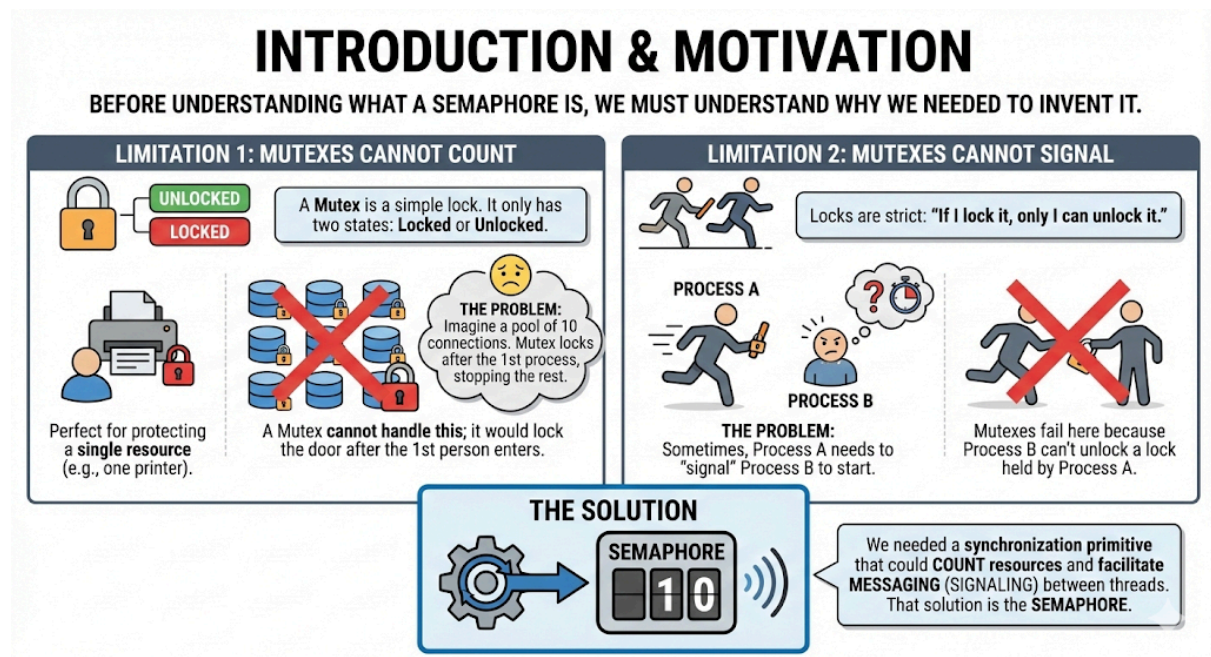


# Lecture 13 NOTES – Process Synchronization IV: Semaphores

## Introduction & Motivation

Before understanding *what* a semaphore is, you must understand *why* we needed to invent it. We already had Mutexes and Spinlocks, so why add another tool?

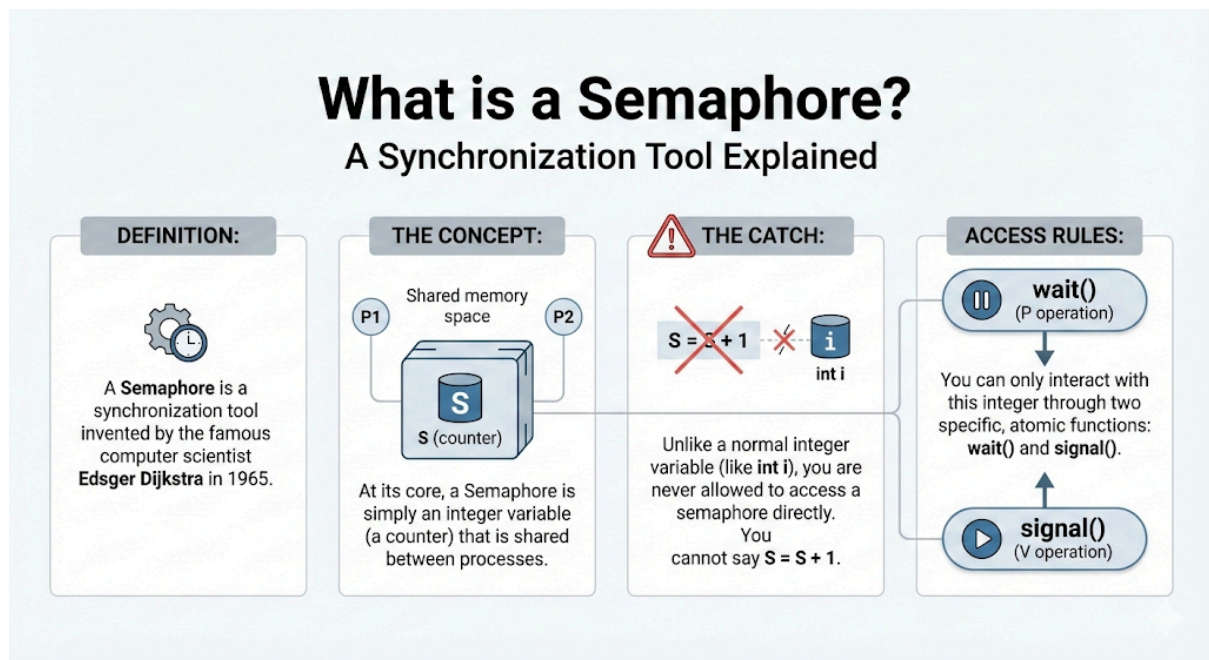
- **Limitation 1: Mutexes cannot Count.**
  - A Mutex is a simple lock. It only has two states: **Locked** or **Unlocked**.
  - This is perfect if you are protecting a single resource (like one printer).
  - **The Problem:** Imagine you have a pool of **10 database connections**. You want to allow 10 processes to work at the same time, but stop the 11th process. A Mutex cannot handle this; it would lock the door after the 1st person enters.



- **Limitation 2: Mutexes cannot Signal.**
  - Locks are strict: "If I lock it, only I can unlock it."
  - **The Problem:** Sometimes, you need **Process A** to do some work and then "signal" **Process B** to start (like a relay race). Mutexes fail here because Process B can't unlock a lock held by Process A.
- **The Solution:** We needed a synchronization primitive that could **count resources** and facilitate **messaging** (signaling) between threads. That solution is the Semaphore.

# What is a Semaphore?

- **Definition:** A Semaphore is a synchronization tool invented by the famous computer scientist **Edsger Dijkstra** in 1965.



- **The Concept:** At its core, a Semaphore is simply an **integer variable** (a counter) that is shared between processes.
- **The Catch:** Unlike a normal integer variable (like `int i`), you are **never** allowed to access a semaphore directly. You cannot say  $S = S + 1$ .
- **Access Rules:** You can only interact with this integer through two specific, atomic functions: **wait()** and **signal()**.

## The Semaphore Abstract Data Type

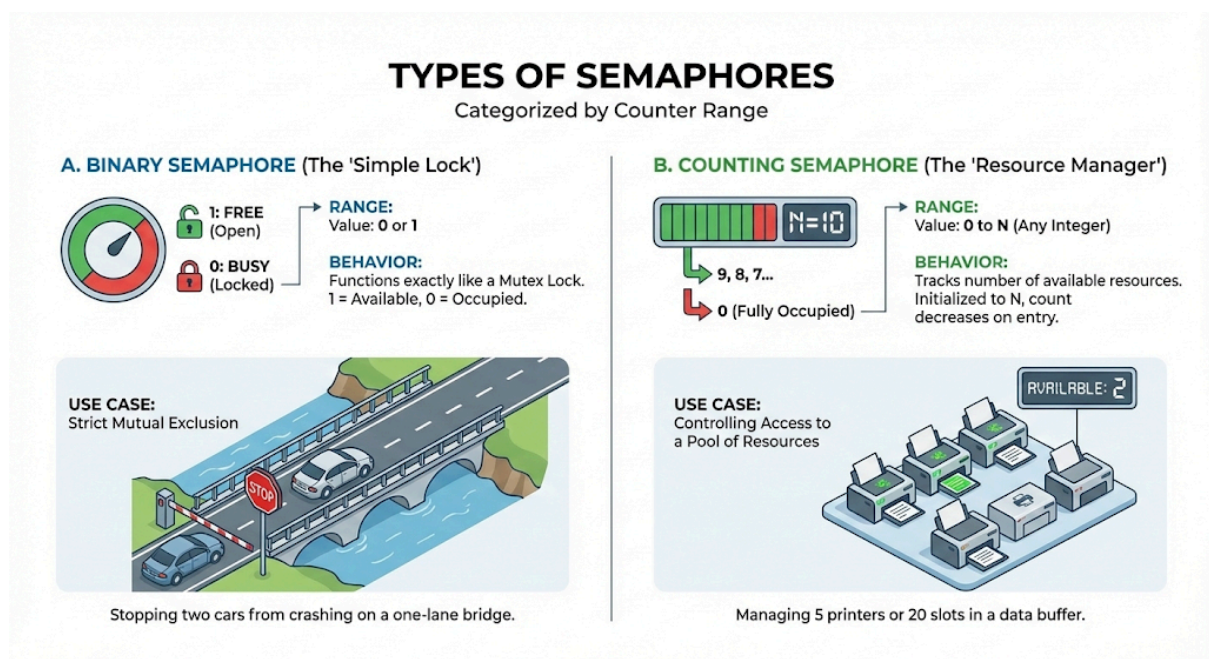
It is not just an integer

```
typedef struct {  
    int value;           // Counter  
    (available resources)  
    queue waiting_list; // List of blocked  
                        threads  
} semaphore;
```



## Types of Semaphores

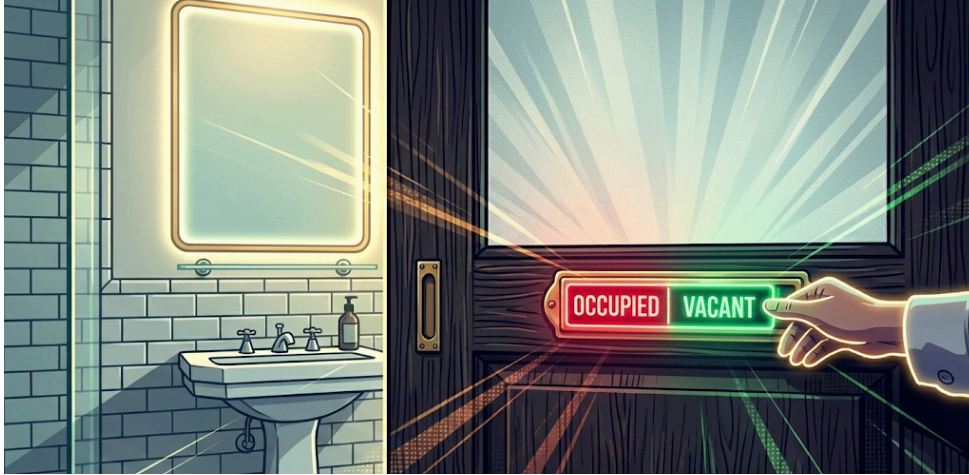
There are two distinct types of semaphores, categorized by how high the counter can go.





## A. Binary Semaphore (The "Simple Lock")

- **Range:** The value can only be **0** or **1**.
- **Behavior:** It functions exactly like a **Mutex Lock**.
  - **1** represents "Free" (Open).
  - **0** represents "Busy" (Locked).
- **Use Case:** Strict Mutual Exclusion. Used when you just want to stop two cars from crashing on a one-lane bridge.



## B. Counting Semaphore (The "Resource Manager")

- **Range:** The value can be any integer from **0** to **N**.
- **Behavior:** It tracks the **number** of available resources.
  - If you initialize it to **N = 10**, it means "10 spots are available."
  - Every time a process enters, the count goes down (**9, 8, 7...**).
  - When the count hits **0**, the resource is fully occupied.
- **Use Case:** Controlling access to a pool of resources, like managing 5 printers or 20 slots in a data buffer.





# Visualizing the Difference



## Binary:

[1] → [0] (Lock) → [1] (Unlock)

## Counting (Init 3):

[3] → [2] → [1] → [0] (Full) → Wait

How is a counting semaphore ( $N=3$ ) different from 3 binary semaphores?

**Answer:** The counting semaphore manages the pool as a single unified group.

08

## Semaphore Operations (The Core Mechanism)

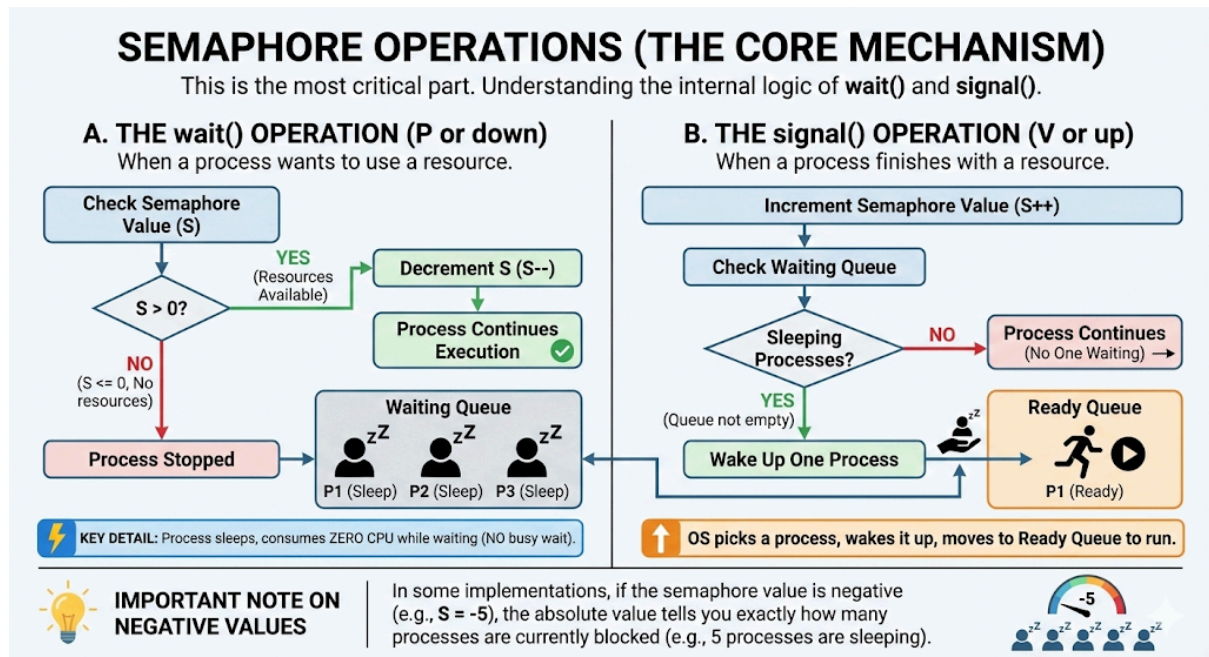
This is the most critical part. You must understand exactly what happens inside these two operations.

**A. The wait() Operation (Also called P or down)** This is the function a process calls when it **wants** to use a resource.

- **The Logic:**

1. **Check:** The function checks the integer value of the semaphore ( $S$ ).

2. **If  $S > 0$  (Resources available):** It decrements the value by 1 ( $S--$ ) and the process continues execution.
  3. **If  $S \leq 0$  (No resources):** The process is **stopped**. It does *not* spin in a loop (busy wait). Instead, the OS moves it to a **Waiting Queue** and puts it to sleep (block).
- **Key Detail:** Since the process sleeps, it consumes **zero CPU** while waiting.



**B. The `signal()` Operation (Also called **V** or **up**)** This is the function a process calls when it is **finished** with a resource.

- **The Logic:**
  1. **Increment:** The function increments the semaphore value ( $S++$ ).
  2. **Check Queue:** It checks if any processes are asleep in the waiting queue.
  3. **Wake Up:** If there are sleeping processes, the OS picks **one** of them, wakes it up, and moves it to the Ready Queue so it can run.

**Important Note on Negative Values:** In some implementations, if the semaphore value is negative (e.g.,  $-5$ ), the absolute value tells you exactly **how many processes are currently blocked** (5 processes are sleeping).



## Signal : Pseudo Code

```
signal(S) {  
    S→value++;  
    if (S→value ≤ 0) {  
        remove_thread_from_queue();  
        wakeup(thread);  
    }  
}
```

### The Rule of Magnitude:

#### If value is negative:

- (e.g., -3), the magnitude (3) equals the number of sleeping threads.

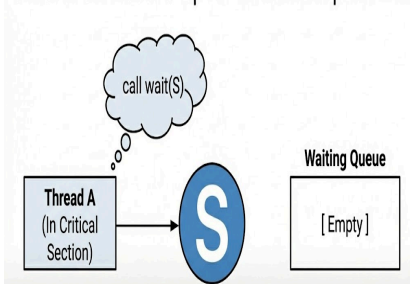
#### If value is zero:

- 0 resources left, but 0 threads waiting. (The boundary case).

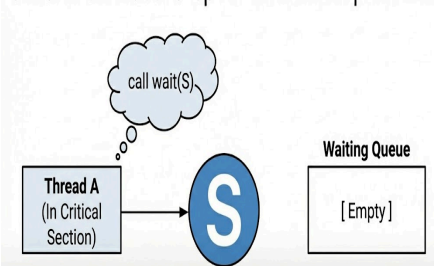
**Crucial Insight:** If value was negative, it means someone was waiting.

## Simulation Trace

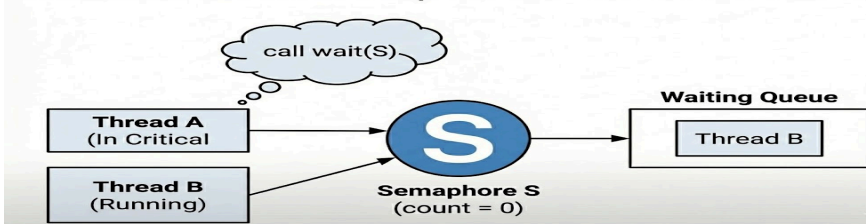
Simulation Trace: Step 2 - Thread A Acquires Lock



Simulation Trace: Step 2 - Thread A Acquires Lock



Simulation Trace: Step 3 - Thread B is Blocked



**Scenario:** Init = 3 (Printers). 5 Threads request access.

### Trace:

Time 0: Val = 3.

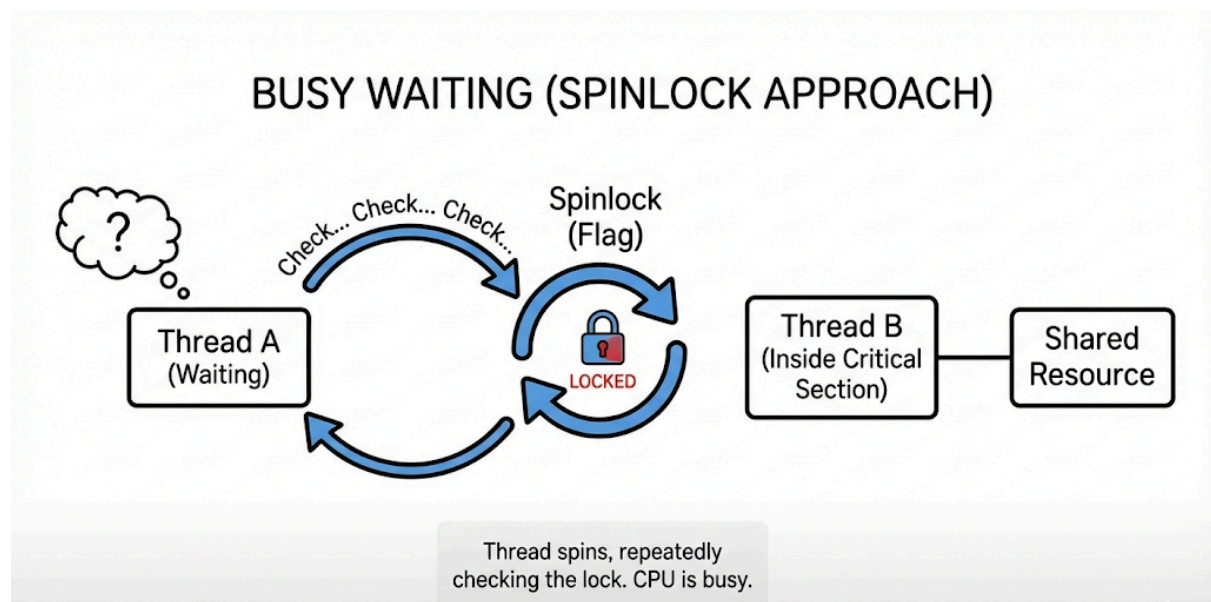
Time 3 (A,B,C wait): Val = 0. (All taken).

Time 4 (D waits): Val = -1. Queue: [D].

Time 5 (E waits): Val = -2. Queue: [D, E].

Time 6 (A signals): Val = -1. D wakes up.

## Busy Waiting (Spring Lock Approach)



**1. The Mechanism: What is it?** Busy waiting is a synchronization technique where a process or thread repeatedly checks a condition to see if it can proceed. Instead of "going to sleep" (blocking) and waiting for the Operating System to wake it up later, the thread remains active on the CPU, spinning in a tight loop.

+1

- **The Logic:** It executes a loop like `while (!resource_is_free) { /* do nothing */ }`.
- **OS State:** The thread stays in the **Running State** the entire time. It does not move to the *Waiting* or *Blocked* queue.

**2. CPU Impact: The Cost** Because the thread never yields the processor, it consumes **100% of the CPU core's capacity** while waiting.

- **Why is this "wasteful"?** The CPU is executing instructions efficiently, but those instructions are "useless" checks. It is effectively burning electricity to

ask "Is it ready yet?" millions of times per second.

- **Analogy for Students:** Imagine waiting for a table at a restaurant. **Busy Waiting** is standing at the hostess stand and asking "Is my table ready?" every 3 seconds. You aren't doing anything else, and you are annoying the hostess (wasting resources).

### 3. Latency: The Benefit If busy waiting is so wasteful, why do we use it? **Speed.**

- **Ultra-Low Latency:** Since the thread is already on the CPU and running, there is zero "wake up" time. As soon as the resource becomes available, the thread grabs it immediately (in nanoseconds).
- **Comparison:** Blocking requires a "Context Switch" (saving the thread's state, putting it to sleep, and restoring it later), which takes microseconds. Spinlocks avoid this overhead.

### 4. Use Case: When to use it? You should only use Busy Waiting for **tiny critical sections** (typically less than 1 microsecond).

- **Good Scenario:** Incrementing a simple integer counter. The work takes 2 nanoseconds. It is faster to spin for 2 nanoseconds than to spend 5 microseconds putting the thread to sleep.
- **Bad Scenario:** Waiting for a database query. If you spin while waiting for a hard drive (which takes milliseconds), you will waste millions of CPU cycles doing nothing.

## Code Example

```
// A simple simulation of Busy Waiting (Spinlock) using an atomic
flag

#include <stdatomic.h>
#include <stdbool.h>

atomic_flag lock = ATOMIC_FLAG_INIT;

void enter_critical_section() {
    // THE SPIN:
    // atomic_flag_test_and_set returns the PREVIOUS value.
    // If it returns true (1), it means it was ALREADY locked.
    // So we loop (spin) until it returns false (0).
```



```

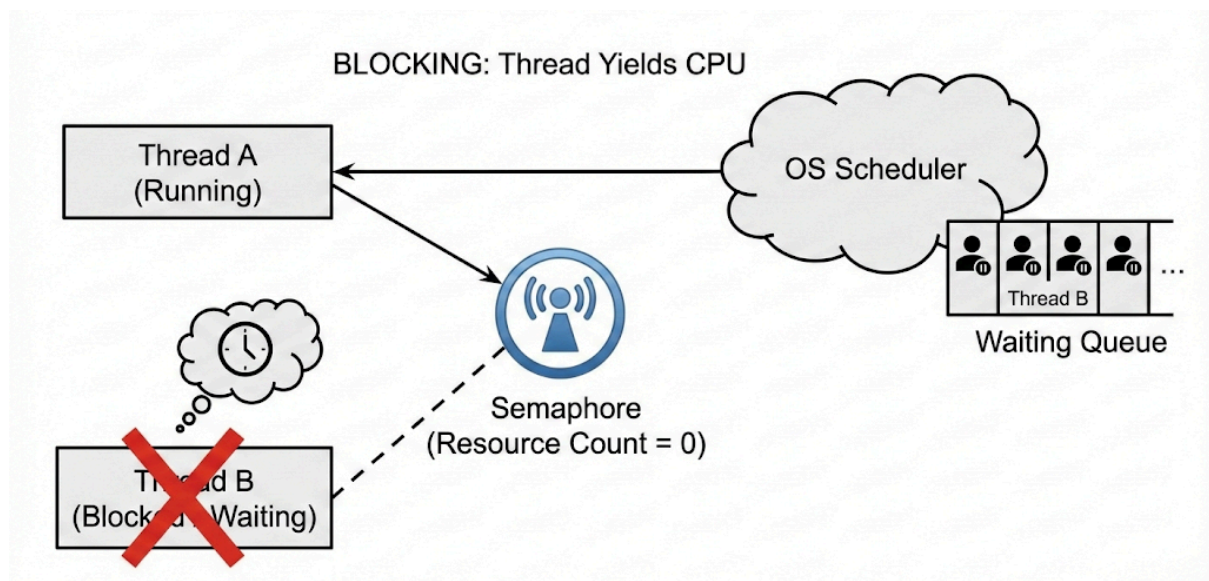
while (atomic_flag_test_and_set(&lock)) {
    // Do nothing. Just spin.
    // This is the "Busy Wait" that burns CPU.
}

// If the loop breaks, we have acquired the lock!
}

void leave_critical_section() {
    // Release the lock
    atomic_flag_clear(&lock);
}

```

## Blocking (The Semaphore Approach)



**1. The Mechanism: What is it?** Blocking is a synchronization technique where a thread voluntarily gives up the CPU if the resource it needs is not available. Instead of spinning in circles (like a Spinlock), the thread effectively tells the Operating System, "I cannot proceed right now, so put me to sleep and wake me up when the resource is ready".

- **The Logic:** When a thread calls `wait()` and finds the semaphore value is 0 (or less, indicating no resources), it executes a system call (often named `block()` or `sleep()`) rather than looping.

- **OS State:** The Operating System changes the thread's state from **Running** to **Waiting** (or Blocked) and moves its Process Control Block (PCB) into a waiting queue associated with that semaphore.

**2. CPU Usage: The Benefit** Because the thread is "asleep," it consumes **zero CPU cycles** while waiting.

- **Efficiency:** The OS Scheduler sees that this thread is blocked and immediately schedules a different, ready thread to run on that CPU core. This maximizes overall system throughput because the CPU is always doing useful work for *someone*, rather than just spinning on a lock.
- **Analogy for Students:** This is like leaving your phone number with the restaurant hostess. You don't stand there repeatedly asking "Is it ready?" (Busy Wait). Instead, you go sit in your car or take a walk (yield the resource). When the table is ready, the hostess calls you (signals/wakes you up).

**3. Latency: The Cost** While efficient for the CPU, blocking introduces **latency** (delay) for the thread itself.

- **Context Switching:** Putting a thread to sleep and waking it up later involves a "Context Switch." The OS must save the current thread's registers and stack, load another thread, and later reload the original thread.
- **The "Expensive" Operation:** This process typically takes **microseconds** (thousands of nanoseconds). If a resource becomes free just 1 nanosecond after you decide to sleep, you have still paid the "penalty" of the full context switch time just to wake back up.

**4. Use Case: When to use it?** Blocking is the preferred method for **long waits** or **high contention** scenarios.

- **Long Waits:** If a thread is waiting for a disk read (I/O) or a database query, the wait could be milliseconds long. Spinning for milliseconds is a huge waste of CPU. Blocking is perfect here because the context switch cost (microseconds) is tiny compared to the I/O wait time.
- **High Contention:** If 50 threads are fighting for 1 lock, a spinlock would have 49 threads burning CPU at 100%. A semaphore puts 49 threads to sleep, leaving the CPU free for the 1 thread doing the actual work.

## Code Example

```
// Pseudo-code for Blocking Logic (Inside the OS Kernel)

void wait(semaphore *S) {
    S->value-- ;
```

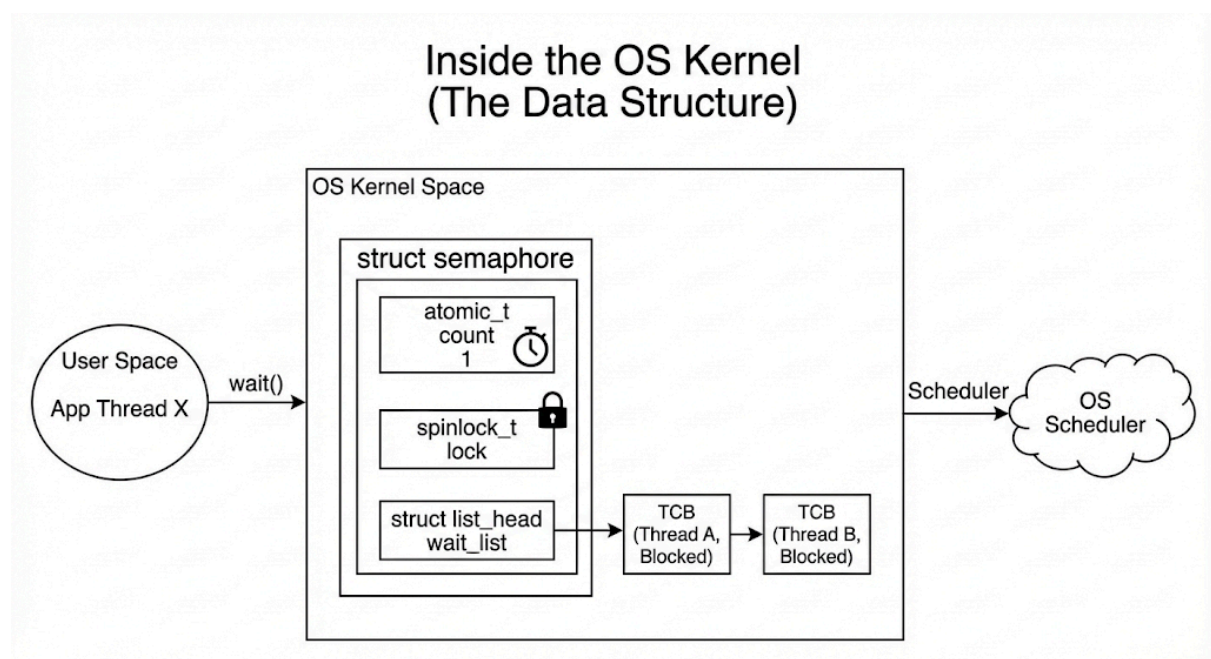
```

if (S→value < 0) {
    // RESOURCE BUSY:
    // 1. Add this process to the semaphore's waiting list
    add_to_queue(S→list, current_process);

    // 2. BLOCK (The Key Step):
    // Tell the Scheduler: "Stop running me."
    // The CPU switches to a different process immediately.
    block();
}
}
// When the thread wakes up later, it resumes execution right here.

```

## Inside the OS Kernel (The Data Structure)



**1. The "Black Box" Revealed** In theory, we often describe a semaphore simply as an integer variable. However, in a real Operating System, a simple integer is not enough to handle complex thread management. A semaphore is actually a robust **Kernel Object**—a complex data structure that lives in the protected memory space of the OS.

**2. Kernel Struct Components** When you initialize a semaphore, the OS allocates a structure (usually called `struct semaphore` or similar) containing three critical components:

- **A. Atomic Integer (count/value):** This is the counter we discuss in theory. It tracks the number of available resources (e.g., "5 printers left").
  - *Constraint:* It must be manipulated **atomically**. The hardware guarantees that reading, decrementing, and writing this value happens in one unbreakable step so that two threads don't modify it simultaneously.
- **B. The Wait Queue (wait\_list):** This is the most important part for blocking. It is a linked list of **Thread Control Blocks (TCBs)**.
  - *Function:* When a thread tries to wait on a semaphore with a value of 0, the OS doesn't just freeze the thread randomly. It takes the thread's TCB (its ID, registers, and state) and adds it to this specific list. This ensures that when we signal later, we know exactly *who* was waiting.
- **C. The Internal Lock (spinlock):** This is a "meta-lock." Since the semaphore is a shared object, multiple threads might try to call `wait()` or `signal()` at the exact same time.
  - *Purpose:* To prevent corruption of the *semaphore structure itself*, the OS uses a tiny, fast spinlock. It locks the structure just long enough to safely decrement the integer or add a node to the linked list.

**3. Why Kernel Space?** Why can't we just build this in our own program's memory (User Space)?

- **Privileged Operations:** The act of "blocking" a thread is not simple. It requires removing the thread from the CPU and placing it into a "Waiting" state in the Scheduler. Modifying the Scheduler's run queue is a **privileged operation** that only the OS Kernel is allowed to perform.
- Therefore, every time you call `sem_wait()`, your program makes a **System Call**, handing control over to the Kernel to perform the heavy lifting safely.

**Kernel Data Structure Visualization:**

```
// A simplified view of what a Semaphore looks like in the Linux
Kernel

typedef struct {
    // 1. The Value: Tracks resources (e.g., 5)
    // 'atomic_t' ensures hardware-level safety.
    atomic_t count;
```



```

// 2. The Queue: Who is sleeping?
// A linked list of tasks (TCBs) waiting for this resource.
struct list_head wait_list;

// 3. The Protection: A tiny lock for the struct itself
// Prevents two CPUs from messing up the 'wait_list' at the same
time.
spinlock_t lock;

} semaphore_t;

```

## The Atomicity Challenge

**1. The Critical Race: What goes wrong?** A semaphore relies entirely on the accuracy of its integer counter (`S->value`). A classic "Race Condition" occurs if two threads try to modify this value at the exact same nanosecond on different CPU cores.

- **The Scenario:** Imagine the semaphore value is `1` (one resource left).
  - **Thread A** calls `wait()`: It reads the value `1`. It prepares to decrement it to `0`.
  - **Thread B** calls `wait()` *simultaneously*: It also reads the value `1` (because Thread A hasn't saved the change yet).
  - **The Crash:** Both threads decrement `1` to `0` and both enter the Critical Section. Now we have two threads using one resource. This violates the fundamental rule of Mutual Exclusion.

**2. The Result: Corrupted State** If atomicity is lost, the semaphore enters a "corrupted state" where the counter no longer matches reality.

- **Phantom Signals:** `signal()` might increment the counter to `1`, but fail to wake up a sleeping thread because the queue pointer wasn't updated safely.
- **Zombie Counts:** The count might read `-1` (implying a thread is waiting), but the waiting queue is actually empty. The next `signal()` calls `wakeup()` on a null pointer, crashing the kernel.

**3. The Fix: Making it "Indivisible"** To solve this, the Operating System must ensure that the entire logic block—checking the value, decrementing it, and updating the queue—happens as **one atomic unit**. No one can interrupt it halfway.

- **Mechanism A: On Single-Core Systems**

- **Disable Interrupts:** The OS momentarily turns off "Interrupts." This stops the Scheduler from context-switching to another thread. The current thread modifies the variable in peace, then re-enables interrupts.

- **Mechanism B: On Multi-Core Systems**

- **Hardware Spinlocks:** Disabling interrupts doesn't stop a *different* CPU core from accessing memory. So, the OS uses a hardware **Spinlock** (using instructions like **Test-and-Set**).
- **The logic:** Before touching the semaphore count, the core acquires the spinlock. Any other core trying to touch that semaphore must spin (wait) until the lock is released.

## The "Check-Then-Act" Bug

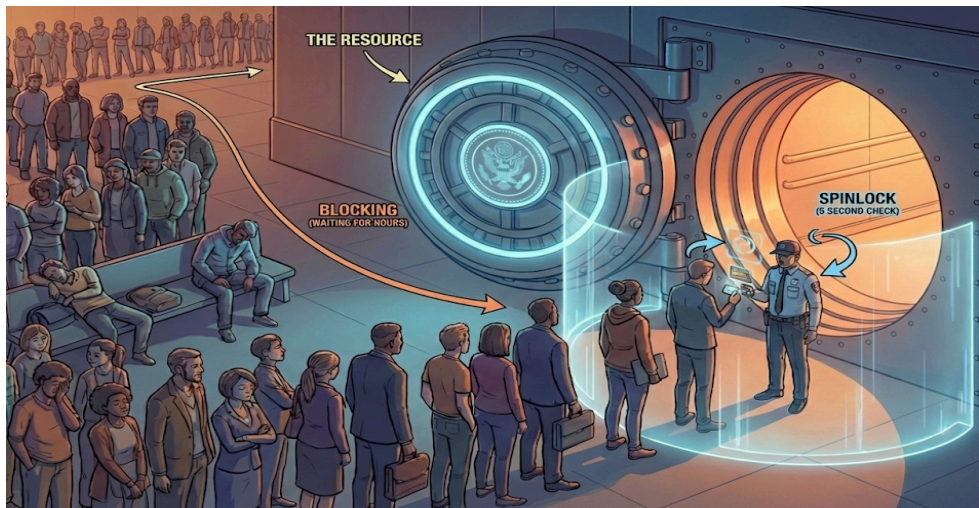
```
// THE BUGGY VERSION (Without Atomicity)
// If two threads run this line at the same time...

void wait(semaphore *S) {
    // <--- RACE CONDITION STARTS HERE
    if (S->value > 0) {
        // Thread A reads 1.
        // Thread B reads 1.
        S->value--;

        // Thread A writes 0.
        // Thread B writes 0.
        // BOTH threads enter the Critical Section!
    }
    // <--- RACE CONDITION ENDS HERE

    else {
        block();
    }
}
```

# The "Spinlock Inside" (Implementation Irony)



## 1. The Paradox

When we introduce Semaphores, we sell them as the solution to the "Busy Waiting" problem. We tell students: *"Spinlocks waste CPU cycles; Semaphores save CPU cycles by sleeping."*

- **The Confusion:** It seems contradictory to then reveal that the Operating System actually uses a Spinlock *inside* the Semaphore implementation.
- **Student Question:** "If we are trying to avoid spinlocks, why are we using one?"

## 2. The Reality: It's About Scope

The answer lies in **what** we are waiting for.

- **User's Critical Section (The Goal):** This is the code the user wants to run (e.g., writing 1GB to a disk or processing a large matrix). This takes **milliseconds or seconds**. We absolutely *cannot* spin for this long.
- **Kernel's Critical Section (The Mechanism):** This is the code the OS runs to update the semaphore's internal state (decrementing **count** and adding a node to the **wait\_queue**). This involves execution of just **10-20 assembly instructions**.
- **Duration:** Executing these instructions takes **nanoseconds**.

## 3. The Solution: A Hybrid Approach

The OS uses a Spinlock to protect the *integrity of the semaphore struct itself*, not to protect the user's resource.

- **The Logic:** It is perfectly acceptable to spin for 5 nanoseconds (to grab the struct lock) in order to avoid a context switch that takes 5 microseconds. The Spinlock is the most efficient tool for this specific, tiny task.

## 4. Key Distinction

You must separate the two different "waits" happening here:

1. **Waiting for Metadata (Spin):** Threads spin briefly to get permission to safely update the integer counter **S->value**.

2. **Waiting for Resource (Block):** Once the counter is updated and the thread realizes the resource is busy, it *stops spinning* and goes to sleep (blocks).

## Internal Implementation Logic

```
// How the OS implements wait() safely using an internal spinlock

void sem_wait(semaphore *S) {
    // 1. ACQUIRE INTERNAL LOCK (The "Spinlock Inside")
    // Protects the struct from other CPUs.
    // This wait is TINY (nanoseconds).
    spinlock_acquire(&S->lock);

    // --- START OF CRITICAL UPDATE ---
    S->value--;

    if (S->value < 0) {
        // Resource is busy. Add myself to the waiting queue.
        add_to_queue(S->wait_list, current_thread);

        // RELEASE LOCK before going to sleep
        spinlock_release(&S->lock);

        // 2. BLOCK (The "Semaphore Sleep")
        // This wait is LONG (milliseconds/seconds).
        // OS puts thread to sleep.
        block();
    } else {
        // Resource acquired!
        spinlock_release(&S->lock);
    }

    // --- END OF CRITICAL UPDATE ---
}
```



# Modern Optimization (The "Two-Phase" Wait)

**1. The Problem: The Cost of Sleeping** In traditional operating systems, a semaphore is strict: if the resource is busy, the thread sleeps immediately. However, putting a thread to sleep is not free.

- **The Overhead:** The process of blocking involves a **Context Switch**—saving the thread's registers, flushing the CPU pipeline, and loading a new thread. This typically takes **5-10 microseconds**.
- **The Waste:** If the resource becomes free just **1 microsecond** later, we have made a terrible mistake. We paid a 10-microsecond penalty to wait for a 1-microsecond event. It's like booking a hotel room just to wait for a table that will be ready in 2 minutes.

**2. The Solution: Two-Phase Locking (Hybrid Approach)** Modern systems, such as **Linux Futexes** (Fast Userspace Mutexes) and the **Windows Dispatcher**, use a smarter "Two-Phase" strategy. They don't give up immediately.

- **Phase 1: The Optimistic Spin** Before calling the heavy OS scheduler, the thread spins on the CPU for a tiny, fixed limit (e.g., 50 to 100 iterations).
  - *Question:* "Is it free yet? Is it free yet?".
  - *Result:* If the resource opens up during this tiny window, the thread grabs it instantly. Zero context switch cost. High performance!
- **Phase 2: The Pessimistic Block** If the resource is *still* busy after the spin limit runs out, the thread admits defeat.
  - *Action:* It calls the OS kernel to yield the CPU and go to sleep (Block).
  - *Result:* The thread stops wasting CPU cycles on a wait that is clearly going to be long.

**3. The Benefit: Best of Both Worlds** This approach gives us the **responsiveness** of a spinlock for short waits (low latency) and the **efficiency** of a semaphore for long waits (no wasted CPU). It dynamically adapts to the situation.

**Hybrid Logic Code Example:**

```
// Pseudo-code for a Two-Phase "Smart" Semaphore
void hybrid_wait(semaphore *S) {
    int spin_count = 0;
    const int SPIN_LIMIT = 50; // Tunable limit
    // --- PHASE 1: SPIN (Optimistic) ---
    while (spin_count < SPIN_LIMIT) {
        if (try_decrement(S)) {
```

```

        return; // Success! No sleeping needed.
    }
    spin_count++;
    cpu_relax(); // Hint to CPU to save power while spinning
}
// --- PHASE 2: BLOCK (Pessimistic) ---
// If we are still here, the wait is likely long.
// Call the heavy kernel function to sleep.
kernel_block(S);
}

```

## Queue Policies & Fairness (Who goes next?)



**1. The Decision Problem** When multiple processes are blocked in the semaphore's waiting queue, and a `signal()` (or `sem_post()`) is called, the Operating System must decide: **which specific thread should be woken up?** This decision depends on the queue management policy implemented by the OS kernel.

**2. Strict FIFO (First-In, First-Out)** In this policy, the queue acts like a standard grocery store line or a printer queue.

- **Mechanism:** The thread that has been waiting the longest is always the one selected to wake up .
- **Terminology:** A semaphore that enforces this strict ordering is often called a **"Strong Semaphore"**.
- **Pro:** It is mathematically fair. It guarantees **No Starvation** because every thread eventually reaches the front of the line.
- **Con:** It ignores urgency. An important system task (like saving a file before a crash) has to wait behind a low-priority background task.

**3. Priority-Based (The "V.I.P." Line)** In this policy, the queue acts like an Emergency Room triage or a VIP club entrance.

- **Mechanism:** When `signal()` is called, the OS scans the queue and wakes up the thread with the highest scheduling priority (e.g., Real-Time Audio or UI rendering), regardless of when it arrived .
- **Terminology:** A semaphore that allows this line-jumping is often called a **"Weak Semaphore"**.
- **Pro:** Responsiveness. Critical tasks get resources immediately.
- **Con: Starvation.** If high-priority threads keep arriving (e.g., a continuous stream of audio data), a low-priority thread (like a backup job) might stay at the back of the line forever and never get the resource .

## The Standard API (POSIX Semaphores)



**1. The Library: `<semaphore.h>`** To use semaphores in Linux or Unix-based systems (like macOS), you must include the POSIX semaphore library. This provides the standard types and functions defined by the IEEE POSIX standard.

- **Data Type: `sem_t`**
  - This is an opaque data type (you don't see inside it). It represents the semaphore object.
  - *Note:* You rarely move semaphores around by value; you almost always pass them by reference (using pointers `&s`).

## 2. The "Big 4" Functions

- **A. `sem_init(&s, pshared, value)`**
  - **Purpose:** Initializes the semaphore before you use it. You cannot use a semaphore without initializing it first.
  - **Parameters:**
    - `&s`: A pointer to your semaphore variable.
    - `pshared`: This determines visibility.
      - Set to **0** for **Threads** (most common). This means the semaphore is shared between threads of the same process.
      - Set to **1** (non-zero) for **Processes**. This means the semaphore is in shared memory and works across different programs.
    - `value`: The initial value of the counter. (e.g., **1** for a lock, **0** for signaling, **N** for a resource pool).
- **B. `sem_wait(&s)`**
  - **Concept:** This is the **P operation** (or "Down").
  - **Behavior:**
    - It tries to decrement the semaphore value.
    - If the value is **> 0**, it decrements it immediately and returns (proceeds).
    - If the value is **0**, it **blocks** (sleeps) until the semaphore becomes available.
- **C. `sem_post(&s)`**
  - **Concept:** This is the **V operation** (or "Up" / "Signal").
  - **Behavior:**
    - It increments the semaphore value.
    - If there are threads waiting in the queue, it wakes one of them up.
    - *Note:* This function never blocks. It is extremely fast.
- **D. `sem_destroy(&s)`**
  - **Purpose:** Cleans up the resources associated with the semaphore.
  - **When to use:** Call this when you are completely finished with the semaphore (e.g., at the end of `main()`). Failing to do this can lead to kernel resource leaks.

**3. Return Values & Error Handling** All these functions return **0** on success and **-1** on failure.

- **The Trap:** System calls can be interrupted. If `sem_wait` returns **-1**, you must check the system variable `errno`.
- **Common Error:** `EINTR` (Interrupted System Call). This happens if a signal (like `Ctrl+C` or a timer) hits the thread while it was waiting. Robust code loops to retry if this happens.

## Code Example

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h> // The Header File

sem_t mutex; // Define the semaphore variable

void* thread_function(void* arg) {
    // 1. WAIT (Lock)
    // Equivalent to P(s). Decrements count. Blocks if 0.
    sem_wait(&mutex);

    // --- CRITICAL SECTION ---
    printf("Entered Critical Section\n");
    // ... do work ...
    // _____

    // 2. POST (Unlock)
    // Equivalent to V(s). Increments count. Wakes waiting threads.
    sem_post(&mutex);

    return NULL;
}

int main() {
    // 3. INIT
    // &mutex : Pointer to the variable
    // 0      : Shared between threads (not processes)
    // 1      : Initial value (1 = Binary Semaphore / Mutex)
    sem_init(&mutex, 0, 1);

    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread_function, NULL);
    pthread_create(&t2, NULL, thread_function, NULL);
}
```



```
pthread_join(t1, NULL);
pthread_join(t2, NULL);

// 4. DESTROY
// Clean up kernel resources
sem_destroy(&mutex);

return 0;
}
```

## The Binary Mutex Pattern



**1. The Objective** The goal of this pattern is **Mutual Exclusion** (Mutex). We want to protect a "Critical Section"—a piece of code that accesses a shared variable (like a counter or a bank balance)—so that only one thread can execute it at a time.

- **The Mechanism:** We use a semaphore that can only take two values: **1** (Unlocked/Available) and **0** (Locked/Busy).

**2. The Implementation Logic** To use a semaphore as a lock, you must follow a strict sequence of operations :

1. **Initialization ( $S=1$ ):** This is the most important step. We initialize the semaphore to **1**. This represents a "single key" sitting on the table.
2. **Acquire (`sem_wait`):** The thread takes the key. The value drops from **1** to **0**. The thread enters the room (Critical Section).

3. **Release (`sem_post`):** The thread puts the key back. The value rises from **0** to **1**. Now, another waiting thread can grab it.

### 3. The "Zero Initialization" Trap (Common Bug)

What happens if you accidentally initialize the semaphore to **0** instead of **1**?

- **The Scenario:** You write `sem_init(&mutex, 0, 0);`.
- **The Action:** The very first thread calls `sem_wait(&mutex)`.
- **The Result:** Since the value is already 0, the thread **blocks immediately**.
- **The Deadlock:** The thread is waiting for *someone* to signal (post) the semaphore. But since *it* is the only thread supposed to run, and it is asleep, no one will ever signal. The program hangs forever.

### 4. The Golden Rule

- **For Locks/Mutexes:** Always initialize to **1**. (Start with the door open).
  - **For Signaling:** Always initialize to **0**. (Start with the thread asleep, waiting for a signal).
- 

### Trace of Execution

```
// Scenario: Two threads trying to increment a shared counter
sem_t mutex;
int counter = 0;

void* thread_work(void* arg) {
    // START: S = 1 (Free)

    // 1. LOCK
    sem_wait(&mutex);
    // State Change: S becomes 0.
    // If another thread arrives now, it blocks because S is 0.

    // --- CRITICAL SECTION ---
    counter++;
    // Only ONE thread is here at a time.
    // _____

    // 2. UNLOCK
```

```

sem_post(&mutex);
// State Change: S becomes 1.
// If there is a waiting thread, it is woken up and S becomes 0
again.

return NULL;
}

int main() {
    // IMPORTANT: Initialize to 1 for Mutual Exclusion
    sem_init(&mutex, 0, 1);
    // ... create threads ...
}

```

## The Resource Pool (Counting Semaphore)



**1. The Scenario** Imagine you have a limited set of identical resources—for example, a pool of **5 Database Connections** or **3 Printers**. You have **50 threads** that want to use them.

- **The Goal:** You want to allow up to **N** threads to work simultaneously.
- **The Restriction:** If the **(N+1)<sup>th</sup>** thread tries to enter, it must wait until one of the first **N** threads leaves.

**2. The Logic** We use a Counting Semaphore initialized to **N** (the total capacity).

- **Initialization (**Init = N**):** We set the semaphore value to the number of available slots.

- *Example:* `sem_init(&pool, 0, 5);` means "5 slots open."
- **Acquire (`sem_wait`):**
  - The first thread calls `wait()`. Value becomes 4. It proceeds.
  - The second thread calls `wait()`. Value becomes 3. It proceeds.
  - ...
  - The fifth thread calls `wait()`. Value becomes 0. It proceeds.
  - **The Block:** The sixth thread (N+1) calls `wait()`. The value is 0. The OS **blocks** this thread and puts it in the waiting queue.
- **Release (`sem_post`):**
  - When one of the active threads finishes, it calls `post()`.
  - The OS wakes up the sleeping sixth thread, allows it to decrement the value, and lets it enter.

**3. Efficiency: Why use a Semaphore?** You might ask, "Why not just use an integer counter and an `if` statement?"

- **The "No Manager" Benefit:** With a semaphore, you do not need to write complex code to manage the queue of waiting users. You don't need a "Manager Thread" checking who is next.
- **Kernel Automation:** The Operating System Kernel handles the queuing, blocking, waking, and fairness automatically. Your code remains clean and simple.

## Resource Pool Code Example

```
// Scenario: Managing a pool of 5 Database Connections
#include <semaphore.h>
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

#define MAX_CONNECTIONS 5

sem_t db_pool; // The Counting Semaphore

void* client_thread(void* id) {
    printf("Client %ld: Waiting for connection ... \n", (long)id);

    // 1. ACQUIRE (Wait)
    // If count > 0, decrement and proceed.
    // If count == 0, BLOCK here.
```

```

sem_wait(&db_pool);

// --- USING RESOURCE ---
printf("Client %ld: CONNECTED! (Using Database)\n", (long)id);
sleep(2); // Simulate work
// -----

printf("Client %ld: Disconnecting.\n", (long)id);

// 2. RELEASE (Signal)
// Increment count. Wake up the next waiting client.
sem_post(&db_pool);

return NULL;
}

int main() {
    // INIT: Set value to N (5)
    // This allows the first 5 threads to pass immediately.
    sem_init(&db_pool, 0, MAX_CONNECTIONS);

    pthread_t threads[10];

    // Create 10 threads (Clients)
    // Only first 5 will run immediately. Next 5 will block.
    for(long i=0; i<10; i++) {
        pthread_create(&threads[i], NULL, client_thread, (void*)i);
    }

    for(long i=0; i<10; i++) {
        pthread_join(threads[i], NULL);
    }

    sem_destroy(&db_pool);
    return 0;
}

```



# Semaphore Implementation (OS Level)

How does the Operating System actually build a semaphore so that it is safe? It creates a specific data structure.

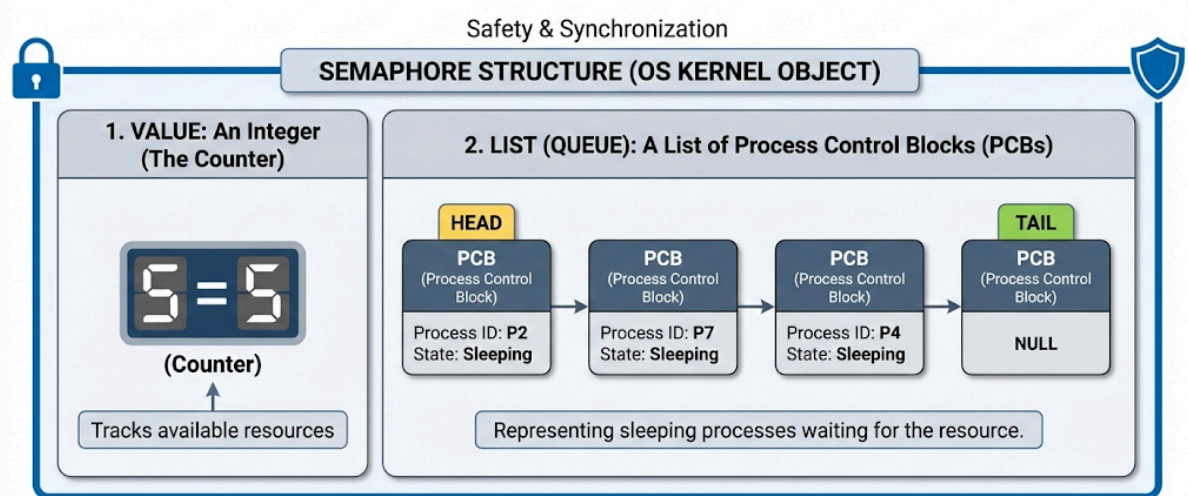
**The Structure:** A semaphore isn't just a number; it is a container holding two things:

1. **Value:** An integer (the counter).
2. **List (Queue):** A list of Process Control Blocks (PCBs) representing the sleeping processes.

## Ensuring Atomicity (Safety): T

### SEMAPHORE IMPLEMENTATION (OS LEVEL): Building a Safe Data Structure

How the Operating System constructs a semaphore for safe synchronization.



This composite data structure ensures atomic operations and process management.

the **wait** and **signal** operations must be **atomic**. This means if one process is changing the number from 5 to 4, no other process can interrupt it halfway.

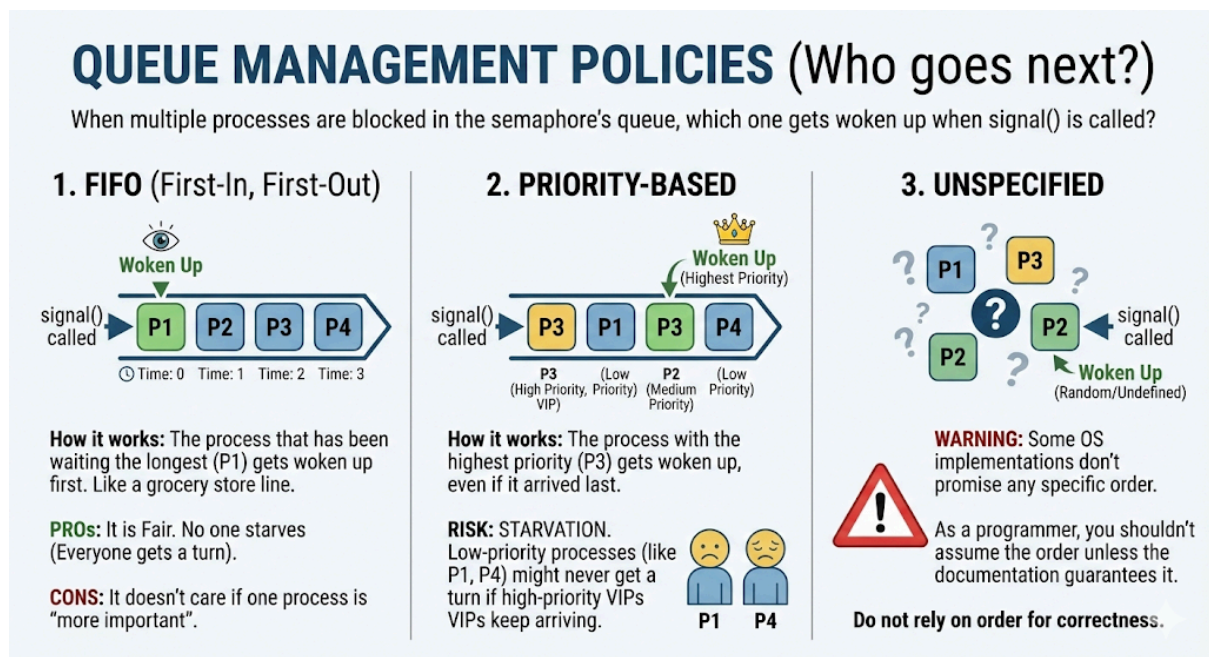
- **On Single-Core Systems:** The OS briefly **disables interrupts**. This stops the scheduler, allowing the current process to update the variable safely, then re-enables interrupts.
- **On Multi-Core Systems:** The OS uses a hardware **Spinlock** (like Test-and-Set).
  - *Step 1:* Acquire Spinlock (locks the semaphore structure).
  - *Step 2:* Update the integer and check the queue (very fast).
  - *Step 3:* Release Spinlock.

- *Why this is okay:* The spinlock is held for only a few nanoseconds, so it doesn't waste CPU like a user-level spinlock would.

## Queue Management Policies (Who goes next?)

When multiple processes are blocked (sleeping) in the semaphore's queue, which one gets woken up when `signal()` is called?

- **FIFO (First-In, First-Out):**
  - **How it works:** The process that has been waiting the longest gets woken up first. Just like a line at a grocery store.
  - **Pros:** It is **Fair**. No one starves.
  - **Cons:** It doesn't care if one process is "more important" than another.



- **Priority-Based:**
  - **How it works:** The process with the highest priority gets woken up, even if it arrived last.
  - **Risk: Starvation.** Low-priority processes might never get a turn if high-priority VIPs keep arriving.
- **Unspecified:**
  - **Warning:** Some OS implementations don't promise any specific order. As a programmer, you shouldn't assume the order unless the documentation guarantees it.

## Advantages of Semaphores

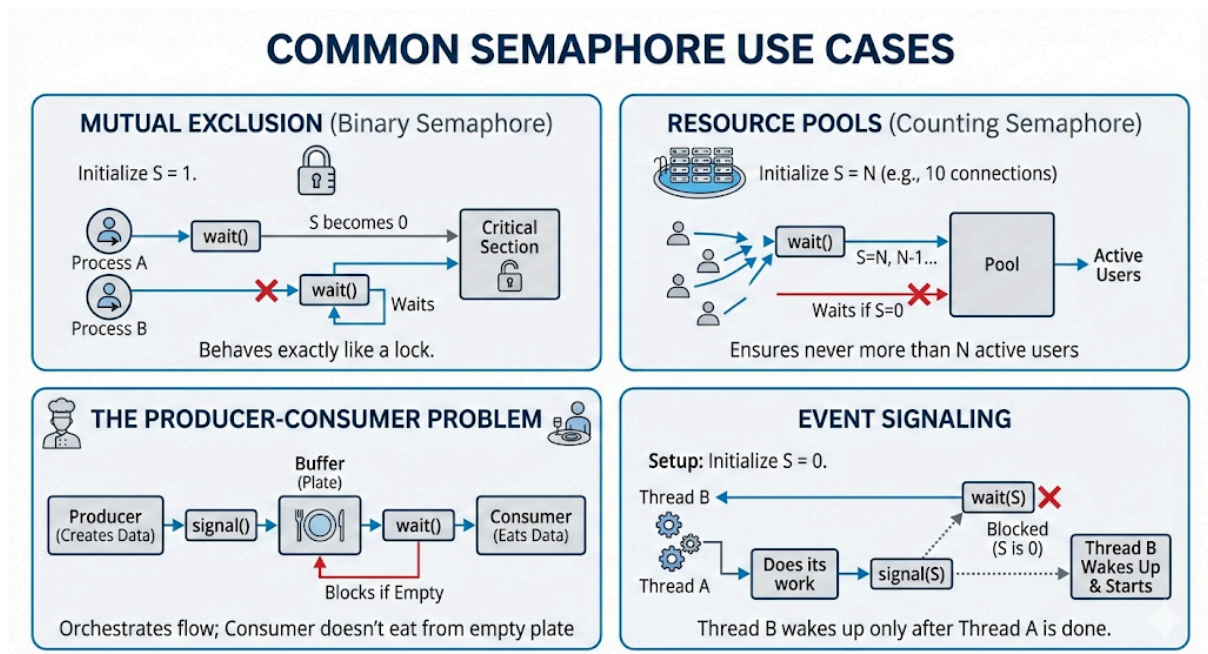
Why do we love them?

1. **No Busy Waiting (Efficiency):**

- This is the biggest win. Unlike Spinlocks that waste CPU cycles asking "Are we there yet?", Semaphores put the waiting process to **sleep**. The CPU is free to do other work.
2. **Resource Counting:**
    - They are the *only* primitive designed to handle multiple instances of a resource (e.g., "We have 3 printers left"). Locks can't do this.
  3. **Machine Independent:**
    - They work in the OS kernel, so they run efficiently on any hardware, whether it's a single core or a massive supercomputer.

## Common Use Cases

- **Mutual Exclusion (Binary Semaphore):**
  - Initialize  $S = 1$ .
  - Process A calls `wait()`,  $S$  becomes 0. Process B waits.
  - Behaves exactly like a lock.



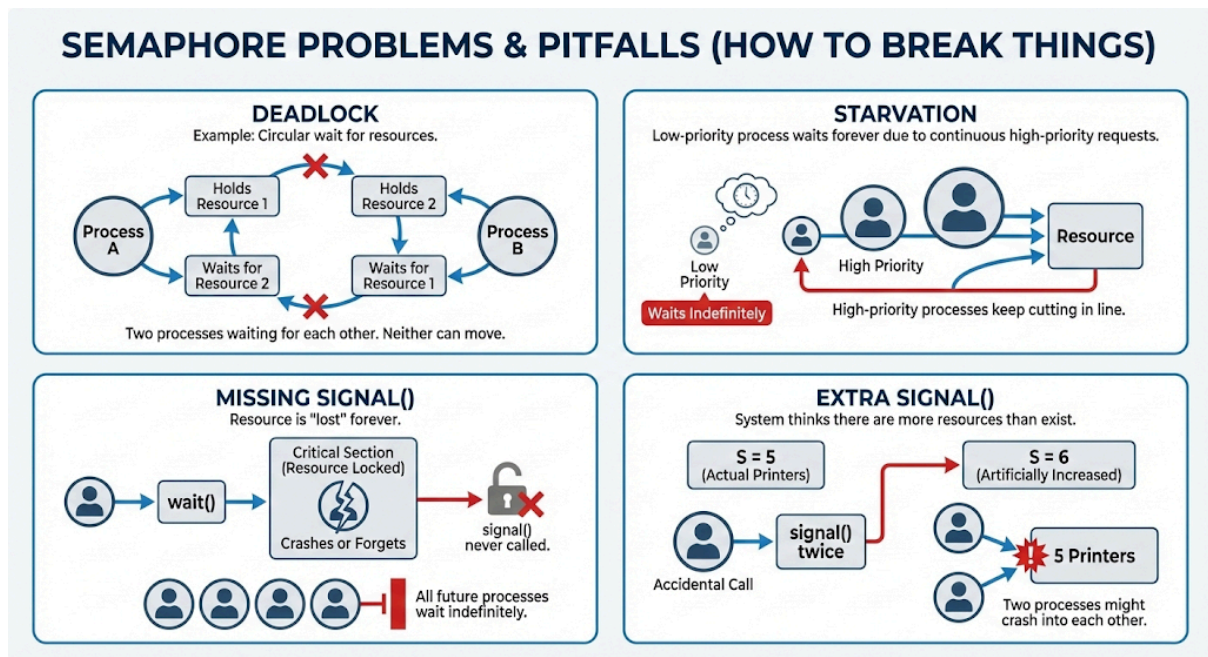
- **Resource Pools (Counting Semaphore):**
  - Initialize  $S = N$  (e.g., 10 database connections).
  - Used to ensure we never have more than  $N$  active users at once.
- **The Producer-Consumer Problem:**
  - A classic problem where one thread creates data and another eats it. Semaphores orchestrate the flow so the Consumer doesn't try to eat from an empty plate.
- **Event Signaling:**
  - **Scenario:** Thread A must finish *before* Thread B starts.
  - **Setup:** Initialize  $S = 0$ .
  - **Thread B:** Calls `wait(S)` immediately (and gets blocked because  $S$  is 0).
  - **Thread A:** Does its work, then calls `signal(S)`.



- **Result:** Thread B wakes up only *after* Thread A is done.

## Problems & Pitfalls (How to break things)

Semaphores are powerful but dangerous. If you mess up the logic, the whole system hangs.



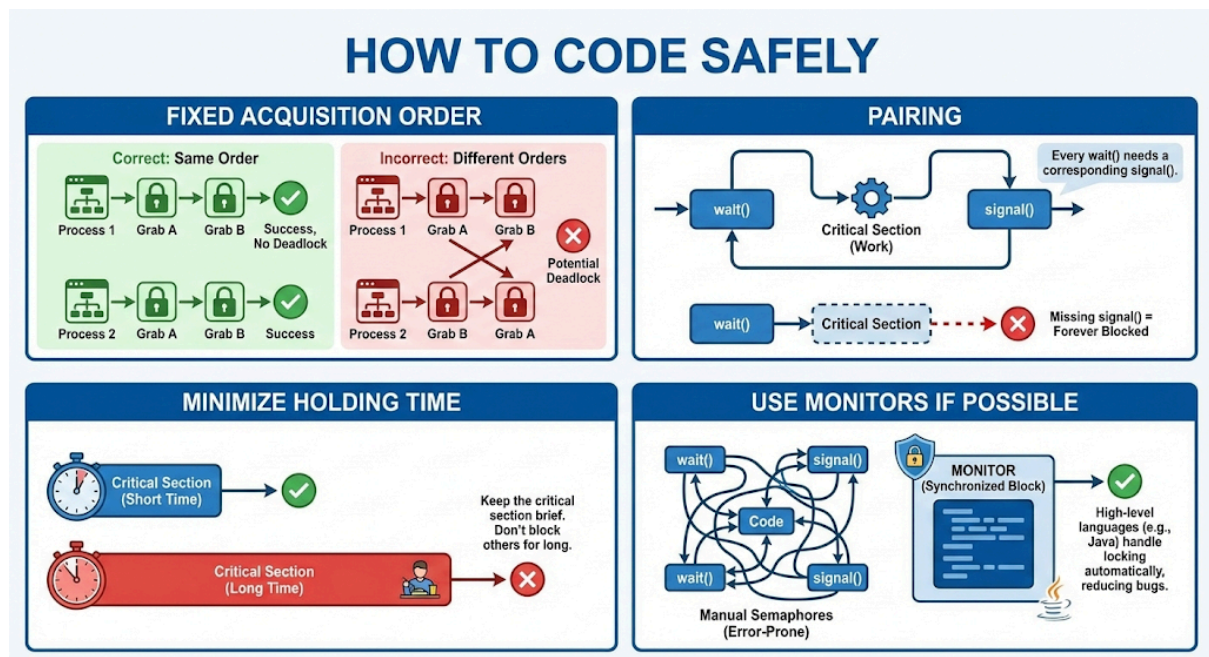
- **Deadlock:**
  - Two processes are waiting for each other to signal. Neither can move.
  - *Example:* Process A holds Resource 1 and waits for 2. Process B holds Resource 2 and waits for 1. They wait forever.
- **Starvation:**
  - In a priority queue, a low-priority process waits indefinitely because new high-priority processes keep cutting in line.
- **Missing `signal()`:**
  - If a process calls `wait()` but crashes or forgets to call `signal()` before finishing, the resource is "lost" forever. All future processes will wait indefinitely.
- **Extra `signal()`:**
  - If you accidentally call `signal()` twice, you artificially increase the counter. Suddenly, the system thinks there are 6 printers when there are only 5. Two processes might crash into each other.

## Best Practices

How to code safely:

1. **Fixed Acquisition Order:** Always grab semaphores in the same order (e.g., always grab A then B). This prevents Deadlock.

2. **Pairing:** Every code path that has a `wait()` must eventually reach a `signal()`.
3. **Minimize Holding Time:** Keep the Critical Section short. Don't go for lunch while holding a semaphore.



4. **Use Monitors if possible:** High-level languages (like Java) have "Monitors" (synchronized blocks) that handle the locking/unlocking for you, reducing bugs.

## Comparison (The Cheat Sheet)

| Feature        | Mutex                                                           | Semaphore                                                                       |
|----------------|-----------------------------------------------------------------|---------------------------------------------------------------------------------|
| Core Concept   | Ownership.                                                      | Signaling.                                                                      |
| Philosophy     | "I have the key. No one else can enter until I leave."          | "I am waiting for a signal (ticket). Anyone can give me a ticket."              |
| Unlocking Rule | <b>Strict.</b> Only the thread that locked the Mutex can unlock | <b>Flexible.</b> Any thread can call <code>signal()</code> . Thread A can wait, |



|                       |                                                                    |                                                                    |
|-----------------------|--------------------------------------------------------------------|--------------------------------------------------------------------|
|                       | it. If Thread A locks it, Thread B cannot unlock it.               | and Thread B can signal to wake it up.                             |
| <b>Internal Value</b> | Strictly <b>0 (Locked)</b> or <b>1 (Unlocked)</b> .                | Can be <b>any integer</b> (0 to N).                                |
| <b>Primary Use</b>    | Protecting a <b>Critical Section</b> (preventing race conditions). | <b>Orchestrating threads</b> (e.g., "Producer, wake up Consumer"). |